

06_model-alert-py

#SQLAlchemy

#PostgreSQL

#python

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ backend/  
    └─ app/  
        └─ models/  
            └─ alert.py
```

El archivo `alert.py` se encuentra dentro de la carpeta de modelos del backend:

```
backend/app/models/
```

Este archivo define el modelo SQLAlchemy `Alert`, que representa la tabla de alertas del laboratorio SIEM MVP.

Una alerta es el resultado generado cuando un evento cumple las condiciones de una regla configurada.

Dentro del flujo del laboratorio, `Alert` aparece después de `Event` y `Rule`:

```
Event  
  ↓  
se evalúa contra  
Rule  
  ↓  
si hay coincidencia, genera  
Alert
```

Este modelo es importante porque representa la salida principal del laboratorio desde el punto de vista de un analista SOC: las alertas que deben revisarse, reconocerse o cerrarse.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,360p' backend/app/models/alert.py
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para leer o transformar texto.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,360p'
```

Indica que se impriman las líneas de la 1 a la 360.

```
backend/app/models/alert.py
```

Es la ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```
from __future__ import annotations

from datetime import datetime

from sqlalchemy import DateTime, ForeignKey, Index, Integer, String, func
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.base import Base


class Alert(Base):
    __tablename__ = "alerts"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)

    rule_id: Mapped[int] = mapped_column(
        Integer,
        ForeignKey("rules.id", ondelete="CASCADE"),
        nullable=False,
        index=True,
    )

    event_id: Mapped[int] = mapped_column(
        Integer,
        ForeignKey("events.id", ondelete="CASCADE"),
        nullable=False,
        index=True,
    )

    title: Mapped[str] = mapped_column(String(200), nullable=False)

    # Agrupación (p.ej. host) para threshold/throttle por "grupo"
    group_key: Mapped[str | None] = mapped_column(String(120), nullable=True, index=True)

    # Ciclo de vida de la alerta (en SOC real esto es básico)
    # open -> alerta nueva
    # ack -> reconocida
    # closed-> cerrada
    status: Mapped[str] = mapped_column(String(16), nullable=False, server_default="open", index=True)

    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        nullable=False,
        server_default=func.now(),
    )

    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        nullable=False,
        server_default=func.now(),
        onupdate=func.now(),
    )

    # Relaciones (no obligatorias para el MVP, pero útiles)
    rule = relationship("Rule")
    event = relationship("Event")

    # Índices adicionales (además de los index=True)
    Index("ix_alerts_rule_id_created_at", Alert.rule_id, Alert.created_at)
    Index("ix_alerts_group_key_created_at", Alert.group_key, Alert.created_at)
```

4 Función general del archivo

El archivo `alert.py` define la estructura de las alertas dentro de la base de datos.

La clase principal es:

```
class Alert(Base):
```

Esta clase hereda de `Base`, la clase declarativa definida en:

```
backend/app/db/base.py
```

Gracias a esta herencia, SQLAlchemy interpreta `Alert` como un modelo ORM.

La tabla asociada se llama:

```
__tablename__ = "alerts"
```

Por tanto, la relación es:

```
Clase Python Alert
      ↓
Tabla PostgreSQL alerts
```

Este modelo almacena información relacionada con:

```
id          → identificador único de la alerta
rule_id     → regla que generó la alerta
event_id    → evento que disparó la alerta
title       → título descriptivo
group_key   → clave de agrupación
status      → estado de la alerta
created_at  → fecha de creación
updated_at  → fecha de última actualización
```

Además, define relaciones ORM con:

```
Rule
Event
```

Esto permite acceder desde una alerta a la regla y evento asociados.

5 Estructura general del archivo

El archivo puede dividirse en seis bloques:

```
from __future__ import annotations
```

Importación futura para anotaciones modernas.

```
from datetime import datetime
```

Importación del tipo `datetime`.

```
from sqlalchemy import DateTime, ForeignKey, Index, Integer, String, func
from sqlalchemy.orm import Mapped, mapped_column, relationship
```

Importaciones principales de SQLAlchemy.

```
from app.db.base import Base
```

Importación de la clase base declarativa.

```
class Alert(Base):  
    ...
```

Definición del modelo `Alert`.

```
Index(...)  
Index(...)
```

Definición de índices adicionales para optimizar consultas.

Visualmente:

```
alert.py  
├─ Importación futura  
├─ Importación datetime  
├─ Importaciones SQLAlchemy  
├─ Importación de Base  
├─ Modelo Alert  
│   ├── __tablename__  
│   ├── id  
│   ├── rule_id  
│   ├── event_id  
│   ├── title  
│   ├── group_key  
│   ├── status  
│   ├── created_at  
│   ├── updated_at  
│   ├── rule  
│   └── event  
└─ Índices adicionales  
    ├── ix_alerts_rule_id_created_at  
    └── ix_alerts_group_key_created_at
```

6 Análisis línea por línea

Importación futura de anotaciones

```
from __future__ import annotations
```

Esta línea activa el comportamiento moderno de Python para anotaciones de tipos.

Permite usar anotaciones de forma más flexible, especialmente en casos como:

```
Mapped[str | None]
```

También evita algunos problemas cuando se usan tipos o referencias que pueden evaluarse más adelante.

Importación de `datetime`

```
from datetime import datetime
```

Esta línea importa `datetime` desde el módulo estándar `datetime`.

En este archivo se usa como anotación de tipo en los campos:

```
created_at: Mapped[datetime]  
updated_at: Mapped[datetime]
```

Estos campos representan fechas y horas asociadas al ciclo de vida de la alerta.

Importación de elementos SQLAlchemy

```
from sqlalchemy import DateTime, ForeignKey, Index, Integer, String, func
```

Esta línea importa varios elementos de SQLAlchemy.

DateTime

```
DateTime
```

Representa una columna de fecha y hora.

Se usa en:

```
created_at  
updated_at
```

Ambos campos almacenan marcas temporales con zona horaria.

ForeignKey

```
ForeignKey
```

Permite definir claves foráneas.

Una clave foránea conecta una tabla con otra.

En este archivo se usa en:

```
ForeignKey("rules.id", ondelete="CASCADE")  
ForeignKey("events.id", ondelete="CASCADE")
```

Esto conecta la tabla `alerts` con las tablas `rules` y `events`.

Index

```
Index
```

Permite crear índices adicionales en la base de datos.

Los índices sirven para acelerar consultas frecuentes.

En este archivo se usa al final:

```
Index("ix_alerts_rule_id_created_at", Alert.rule_id, Alert.created_at)  
Index("ix_alerts_group_key_created_at", Alert.group_key, Alert.created_at)
```

Integer

```
Integer
```

Representa una columna de tipo entero.

Se usa en:

```
id
rule_id
event_id
```

String

```
String
```

Representa una cadena de texto con longitud limitada.

Se usa en:

```
title
group_key
status
```

func

```
func
```

Permite usar funciones SQL desde SQLAlchemy.

En este archivo se usa:

```
func.now()
```

Esto representa la función SQL `NOW()` , usada para asignar fechas automáticamente.

Importación ORM

```
from sqlalchemy.orm import Mapped, mapped_column, relationship
```

Esta línea importa tres elementos del ORM moderno de SQLAlchemy.

Mapped

```
Mapped
```

Se usa para anotar atributos que representan columnas o relaciones mapeadas.

Ejemplo:

```
id: Mapped[int]
```

mapped_column

```
mapped_column
```

Se usa para definir columnas de una tabla.

Ejemplo:

```
id: Mapped[int] = mapped_column(Integer, primary_key=True)
```

relationship

```
relationship
```

Permite definir relaciones ORM entre modelos.

En este archivo se usa en:

```
rule = relationship("Rule")
event = relationship("Event")
```

Esto permite que una alerta pueda acceder al objeto `Rule` o `Event` relacionado.

Importación de `Base`

```
from app.db.base import Base
```

Esta línea importa la clase `Base` definida en:

```
backend/app/db/base.py
```

`Alert` hereda de esta clase:

```
class Alert(Base):
```

Gracias a esto, SQLAlchemy registra `Alert` como modelo ORM.

Definición de la clase `Alert`

```
class Alert(Base):
```

Esta línea define la clase `Alert`.

Desglose:

```
class
```

Palabra clave de Python para definir una clase.

```
Alert
```

Nombre de la clase.

Representa una alerta generada por el laboratorio.

```
(Base)
```

Indica que la clase hereda de `Base`.

Esto convierte `Alert` en un modelo SQLAlchemy.

Nombre de la tabla

```
__tablename__ = "alerts"
```

Esta línea indica el nombre de la tabla asociada al modelo.

```
__tablename__
```

Es un atributo especial usado por SQLAlchemy.

```
"alerts"
```

Es el nombre real de la tabla en PostgreSQL.

La relación es:

```
Alert → alerts
```

Campo `id`

```
id: Mapped[int] = mapped_column(Integer, primary_key=True)
```

Esta línea define la columna `id`.

Desglose:

```
id
```

Nombre del atributo y de la columna.

```
Mapped[int]
```

Indica que es una columna mapeada de tipo entero.

```
mapped_column(Integer, primary_key=True)
```

Define la columna como entero y clave primaria.

La clave primaria identifica de forma única cada alerta.

Campo `rule_id`

```
rule_id: Mapped[int] = mapped_column(
    Integer,
    ForeignKey("rules.id", ondelete="CASCADE"),
    nullable=False,
    index=True,
)
```

Este bloque define la columna `rule_id`.

`rule_id` indica qué regla generó la alerta.

Desglose:

```
rule_id: Mapped[int]
```

Indica que el campo es un entero mapeado por SQLAlchemy.

```
Integer
```

Define el tipo de columna como entero.

```
ForeignKey("rules.id", ondelete="CASCADE")
```

Define una clave foránea hacia la tabla `rules`, columna `id`.

Esto significa:

```
alerts.rule_id → rules.id
```


Es decir, cada alerta está asociada a una regla concreta.

`ondelete="CASCADE" en rule_id`

```
ondelete="CASCADE"
```

Indica que si se elimina una regla, se eliminarán también las alertas asociadas a esa regla.

Conceptualmente:

```
Eliminar Rule
  ↓
eliminar Alert relacionadas
```

Esto evita dejar alertas huérfanas apuntando a una regla que ya no existe.

`nullable=False`

```
nullable=False
```

Indica que toda alerta debe estar asociada obligatoriamente a una regla.

No puede existir una alerta sin `rule_id`.

`index=True`

```
index=True
```

Crea un índice sobre la columna `rule_id`.

Esto mejora el rendimiento de consultas que filtren alertas por regla.

Ejemplo conceptual:

```
SELECT * FROM alerts WHERE rule_id = 1;
```

Campo `event_id`

```
event_id: Mapped[int] = mapped_column(
    Integer,
    ForeignKey("events.id", ondelete="CASCADE"),
    nullable=False,
    index=True,
)
```

Este bloque define la columna `event_id`.

`event_id` indica qué evento provocó la alerta.

Desglose:

```
event_id: Mapped[int]
```

Indica que es un entero mapeado.

```
Integer
```

Tipo entero.

```
ForeignKey("events.id", ondelete="CASCADE")
```

Clave foránea hacia la tabla `events`, columna `id`.

Relación:

```
alerts.event_id → events.id
```

Cada alerta está asociada a un evento concreto.

`ondelete="CASCADE" en event_id`

```
ondelete="CASCADE"
```

Indica que si se elimina un evento, también se eliminarán las alertas asociadas a ese evento.

Conceptualmente:

```
Eliminar Event
  ↓
eliminar Alert relacionadas
```

`nullable=False`

```
nullable=False
```

Indica que toda alerta debe estar asociada a un evento.

No puede existir una alerta sin `event_id`.

`index=True`

```
index=True
```

Crea un índice sobre `event_id`.

Esto mejora las consultas que busquen alertas asociadas a un evento concreto.

Campo `title`

```
title: Mapped[str] = mapped_column(String(200), nullable=False)
```

Esta línea define la columna `title`.

Representa el título o descripción breve de la alerta.

Desglose:

```
title
```

Nombre del atributo y de la columna.

```
Mapped[str]
```

Indica que el valor esperado es una cadena.

```
String(200)
```

Cadena de texto con longitud máxima de 200 caracteres.

```
nullable=False
```

Campo obligatorio.

Ejemplo conceptual:

```
High severity event detected
Multiple failed login attempts
Suspicious activity detected
```

Comentario sobre agrupación

```
# Agrupación (p.ej. host) para threshold/throttle por "grupo"
```

Este comentario explica la finalidad del campo `group_key`.

`group_key` sirve para agrupar alertas por algún criterio común.

Por ejemplo:

```
host
IP
usuario
servicio
tipo de evento
```

Esto es útil para reglas de threshold o throttle, donde no solo importa la regla, sino también el grupo sobre el que se está aplicando.

Campo `group_key`

```
group_key: Mapped[str | None] = mapped_column(String(120), nullable=True, index=True)
```

Esta línea define la columna `group_key`.

Desglose:

```
group_key
```

Nombre del atributo y de la columna.

```
Mapped[str | None]
```

Indica que puede ser una cadena o `None`.

Esta sintaxis usa el operador `|`, equivalente moderno a:

```
Optional[str]
```

```
String(120)
```

Cadena de texto con longitud máxima de 120 caracteres.

```
nullable=True
```

Permite valores nulos.

```
index=True
```

Crea un índice sobre la columna.

Función de `group_key`

El campo `group_key` permite agrupar alertas.

Ejemplo conceptual:

```
group_key = "192.168.1.10"  
group_key = "admin"  
group_key = "host-web-01"
```

Esto permite analizar alertas por origen o entidad afectada.

También permite aplicar lógica de threshold o throttle por grupo.

Comentarios sobre ciclo de vida

```
# Ciclo de vida de la alerta (en SOC real esto es básico)  
# open  -> alerta nueva  
# ack   -> reconocida  
# closed-> cerrada
```

Estos comentarios explican los posibles estados de una alerta.

```
open
```

Alerta nueva o abierta.

```
ack
```

Alerta reconocida. En un SOC, esto significa que un analista ya la ha visto o aceptado para revisión.

```
closed
```

Alerta cerrada. Indica que ya se ha gestionado o descartado.

Este ciclo de vida es básico en sistemas de gestión de alertas.

Campo `status`

```
status: Mapped[str] = mapped_column(String(16), nullable=False, server_default="open", index=True)
```

Esta línea define la columna `status`.

Representa el estado actual de la alerta.

Desglose:

```
status
```

Nombre del atributo y de la columna.

```
Mapped[str]
```

Indica que el valor esperado es una cadena.

```
String(16)
```

Cadena con longitud máxima de 16 caracteres.

```
nullable=False
```

El estado no puede ser nulo.

```
server_default="open"
```

La base de datos asignará automáticamente el valor `open` si no se indica otro.

```
index=True
```

Crea un índice sobre la columna `status`.

Esto mejora consultas como:

```
SELECT * FROM alerts WHERE status = 'open';
```

Campo `created_at`

```
created_at: Mapped[datetime] = mapped_column(
    DateTime(timezone=True),
    nullable=False,
    server_default=func.now(),
)
```

Este bloque define la columna `created_at`.

Representa la fecha y hora en la que se creó la alerta.

Nombre y tipo mapeado

```
created_at: Mapped[datetime]
```

Indica que el campo es una columna mapeada y que su tipo Python esperado es `datetime`.

Tipo de columna

```
DateTime(timezone=True)
```

Indica que la columna almacena fecha y hora con zona horaria.

Restricción `nullable=False`

```
nullable=False
```

Indica que el campo no puede ser nulo.

Toda alerta debe tener fecha de creación.

Valor por defecto

```
server_default=func.now()
```

Indica que PostgreSQL asignará automáticamente la hora actual al crear la alerta.

`func.now()` representa la función SQL `NOW()` .

Campo `updated_at`

```
updated_at: Mapped[datetime] = mapped_column(
    DateTime(timezone=True),
    nullable=False,
    server_default=func.now(),
    onupdate=func.now(),
)
```

Este bloque define la columna `updated_at` .

Representa la fecha y hora de la última actualización de la alerta.

Por ejemplo, cuando se cambia el estado de una alerta de:

```
open → ack
ack → closed
```

este campo debería actualizarse.

Tipo de `columna`

```
DateTime(timezone=True)
```

Fecha y hora con zona horaria.

Valor inicial por defecto

```
server_default=func.now()
```

Cuando se crea la alerta, PostgreSQL asigna la hora actual.

Actualización automática

```
onupdate=func.now()
```

Indica que SQLAlchemy debe actualizar este campo cuando se modifique el registro.

Esto permite saber cuándo fue la última vez que se modificó la alerta.

Comentario sobre relaciones

```
# Relaciones (no obligatorias para el MVP, pero útiles)
```

Este comentario introduce las relaciones ORM.

Indica que estas relaciones no son imprescindibles para que el MVP funcione, pero facilitan trabajar con objetos relacionados.

Relación con `Rule`

```
rule = relationship("Rule")
```

Esta línea define una relación ORM entre `Alert` y `Rule` .

Desglose:

```
rule
```

Nombre del atributo de relación.

```
relationship("Rule")
```

Indica que esta relación apunta al modelo `Rule`.

Gracias a esta relación, desde una alerta se podría acceder a la regla asociada:

```
alert.rule
```

Esto es más cómodo que consultar manualmente la tabla `rules` usando `rule_id`.

Relación con `Event`

```
event = relationship("Event")
```

Esta línea define una relación ORM entre `Alert` y `Event`.

Gracias a esta relación, desde una alerta se podría acceder al evento que la generó:

```
alert.event
```

Esto permite navegar entre objetos ORM.

La relación conceptual es:

```
Alert
├─ rule → Rule
└─ event → Event
```

Comentario sobre índices adicionales

```
# Índices adicionales (además de los index=True)
```

Este comentario indica que, además de los índices definidos directamente en columnas con `index=True`, se crean índices compuestos adicionales.

Un índice compuesto incluye más de una columna.

Sirve para acelerar consultas que filtran u ordenan usando esas columnas juntas.

Índice `ix_alerts_rule_id_created_at`

```
Index("ix_alerts_rule_id_created_at", Alert.rule_id, Alert.created_at)
```

Esta línea crea un índice compuesto sobre:

```
rule_id
created_at
```

Nombre del índice:

```
ix_alerts_rule_id_created_at
```

Este índice puede mejorar consultas como:

```
SELECT *
FROM alerts
WHERE rule_id = 1
ORDER BY created_at DESC;
```

También puede ser útil para lógica relacionada con throttle o threshold por regla.

Índice `ix_alerts_group_key_created_at`

```
Index("ix_alerts_group_key_created_at", Alert.group_key, Alert.created_at)
```

Esta línea crea un índice compuesto sobre:

```
group_key
created_at
```

Nombre del índice:

```
ix_alerts_group_key_created_at
```

Este índice puede mejorar consultas como:

```
SELECT *
FROM alerts
WHERE group_key = '192.168.1.10'
ORDER BY created_at DESC;
```

También puede ser útil cuando se agrupan alertas por entidad o se revisan alertas recientes de un grupo concreto.

Resultado final del archivo

Después de cargar este archivo, SQLAlchemy dispone del modelo:

```
Alert
```

Este modelo representa la tabla:

```
alerts
```

con los campos:

```
id
rule_id
event_id
title
group_key
status
created_at
updated_at
```

También dispone de relaciones ORM:

```
rule
event
```

Y define índices adicionales:

```
ix_alerts_rule_id_created_at
```


7 Relación con el flujo técnico del laboratorio

El modelo `Alert` representa la salida principal del sistema.

La relación técnica sería:

```
Event
  ↓
se evalúa contra
Rule
  ↓
si coincide
  ↓
se crea Alert
  ↓
se almacena en PostgreSQL
  ↓
se consulta desde API o frontend
```

Dentro del flujo general del laboratorio:

```
POST /ingest
  ↓
se crea Event
  ↓
se consultan Rules activas
  ↓
si una regla coincide
  ↓
se crea Alert
  ↓
GET /alerts permite consultarla
  ↓
PATCH/PUT puede actualizar su status
```

El modelo `Alert` permite representar alertas con ciclo de vida básico:

```
open → ack → closed
```

Esto acerca el MVP a una lógica habitual de herramientas SOC, donde las alertas no solo se generan, sino que también se gestionan.

8 Errores típicos o puntos importantes

Toda alerta depende de una regla y de un evento

Los campos:

```
rule_id
event_id
```

tienen:

```
nullable=False
```

Por tanto, no puede existir una alerta sin regla o sin evento asociado.

`onDelete="CASCADE"` elimina alertas relacionadas

Si se elimina una regla o evento, las alertas asociadas también pueden eliminarse por cascada.

Esto evita registros huérfanos, pero también implica que borrar reglas o eventos puede borrar historial de alertas.

status tiene valor por defecto

El campo `status` usa:

```
server_default="open"
```

Por tanto, una alerta nueva se crea como abierta si no se indica otro estado.

updated_at usa onupdate

El campo:

```
onupdate=func.now()
```

permite actualizar la fecha cuando se modifica el registro desde SQLAlchemy.

Es útil para saber cuándo se cambió por última vez el estado de la alerta.

group_key puede ser nulo

El campo:

```
group_key
```

permite valores nulos.

Esto significa que no todas las alertas tienen que estar agrupadas.

Índices para consultas frecuentes

El modelo define índices simples con `index=True` y también índices compuestos con `Index`.

Esto mejora consultas habituales como:

- alertas por regla
- alertas por evento
- alertas por estado
- alertas por group_key
- alertas recientes por regla
- alertas recientes por grupo

Relaciones ORM no sustituyen las claves foráneas

Las relaciones:

```
rule = relationship("Rule")
event = relationship("Event")
```

facilitan navegar entre objetos, pero la relación real en la base de datos la definen:

```
ForeignKey("rules.id")
ForeignKey("events.id")
```

9 Comandos útiles relacionados

Comprobar que el modelo se puede importar:

```
docker exec -it siem-api python -c "from app.models.alert import Alert; print(Alert)"
```

Comprobar el nombre de la tabla:

```
docker exec -it siem-api python -c "from app.models.alert import Alert; print(Alert.__tablename__)"
```

Comprobar columnas del modelo:

```
docker exec -it siem-api python -c "from app.models.alert import Alert; print(Alert.__table__.columns.keys())"
```

Comprobar índices del modelo:

```
docker exec -it siem-api python -c "from app.models.alert import Alert; print(Alert.__table__.indexes)"
```

Comprobar estructura de la tabla alerts :

```
docker exec -it siem-db psql -U siem -d siem -c "\d alerts"
```

Consultar alertas existentes:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT id, rule_id, event_id, title, group_key, status, created_at, updated_at FROM alerts LIMIT 10;"
```

Contar alertas:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT COUNT(*) FROM alerts;"
```

Consultar alertas abiertas:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT id, title, status, created_at FROM alerts WHERE status = 'open' ORDER BY created_at DESC;"
```

Consultar alertas por estado:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT status, COUNT(*) FROM alerts GROUP BY status ORDER BY COUNT(*) DESC;"
```

Consultar alertas por group_key :

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT group_key, COUNT(*) FROM alerts WHERE group_key IS NOT NULL GROUP BY group_key ORDER BY COUNT(*) DESC;"
```

Consultar alertas por regla:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT rule_id, COUNT(*) FROM alerts GROUP BY rule_id ORDER BY COUNT(*) DESC;"
```

Consultar alertas con datos de evento y regla mediante JOIN:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT a.id, a.title, a.status, r.name AS rule_name, e.source, e.severity, e.message FROM alerts a JOIN rules r ON a.rule_id = r.id JOIN events e ON a.event_id = e.id LIMIT 10;"
```